Chapter ML:I

I. Introduction

- Examples of Learning Tasks
- Specification of Learning Tasks
- Elements of Machine Learning
- Comparative Syntax Overview
- Classification Approaches

© STEIN/LETTMANN 2021

Examples of Learning Tasks

Car Shopping Guide

?

What criteria form the basis of a decision?

© STEIN/LETTMANN 2021

Examples of Learning Tasks

Risk Analysis for Credit Approval

Customer 1	
house owner	yes
income (p.a.)	51 000 EUR
repayment (p.m.)	1 000 EUR
credit period	7 years
SCHUFA entry	no
age	37
married	yes
...	

...

Customer n	
house owner	no
income (p.a.)	55 000 EUR
repayment (p.m.)	1 200 EUR
credit period	8 years
SCHUFA entry	no
age	?
married	yes
...	

© STEIN/LETTMANN 2021

Examples of Learning Tasks

Risk Analysis for Credit Approval

Customer 1	
house owner	yes
income (p.a.)	51 000 EUR
repayment (p.m.)	1 000 EUR
credit period	7 years
SCHUFA entry	no
age	37
married	yes
...	

...

Customer n	
house owner	no
income (p.a.)	55 000 EUR
repayment (p.m.)	1 200 EUR
credit period	8 years
SCHUFA entry	no
age	?
married	yes
...	

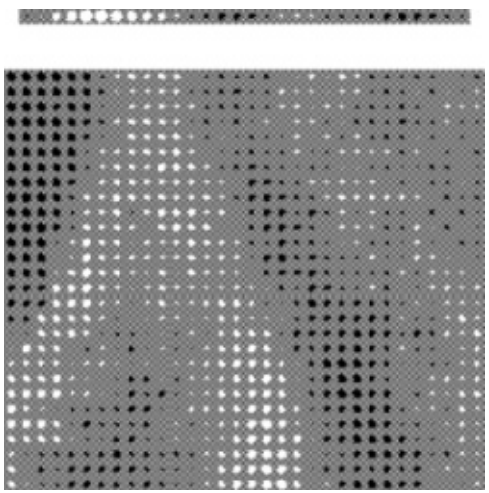
Learned rules:

```
IF    ( income>40 000 AND credit_period<3 )  OR  house_owner=yes
THEN  credit_approval=yes

IF    SCHUFA_entry=yes  OR  ( income<20 000 AND repayment>800 )
THEN  credit_approval=no

...
```

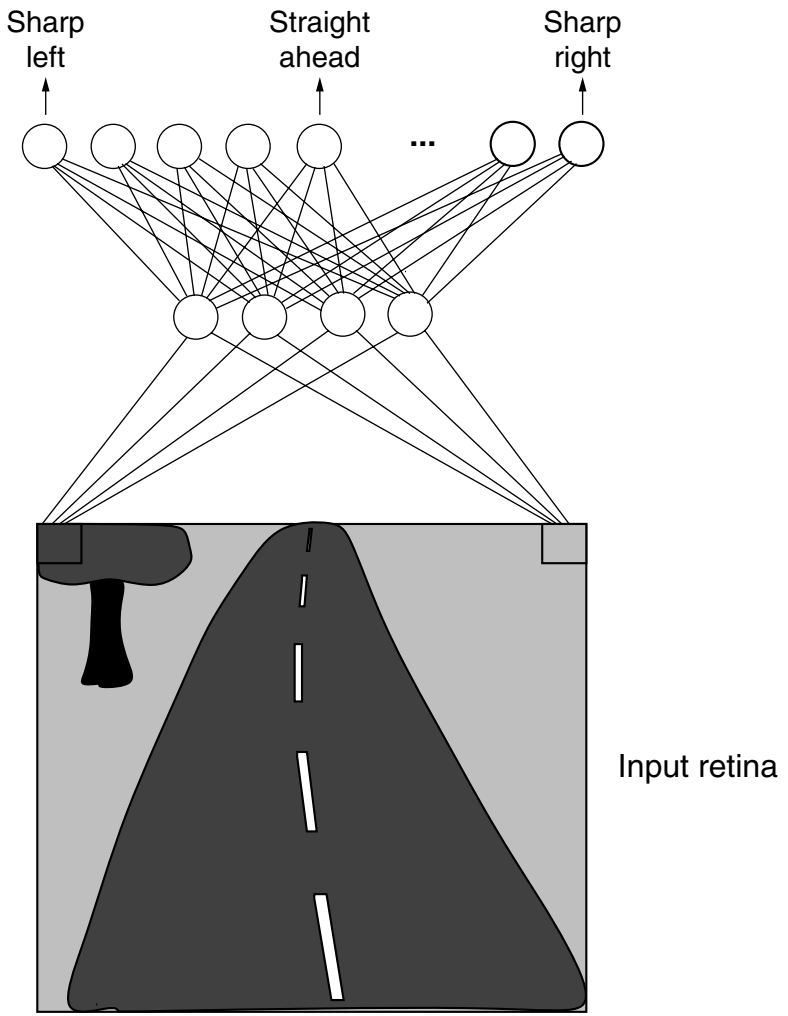

© STEIN/LETTMANN 2021

Examples of Learning Tasks

Image Analysis [Mitchell 1997]



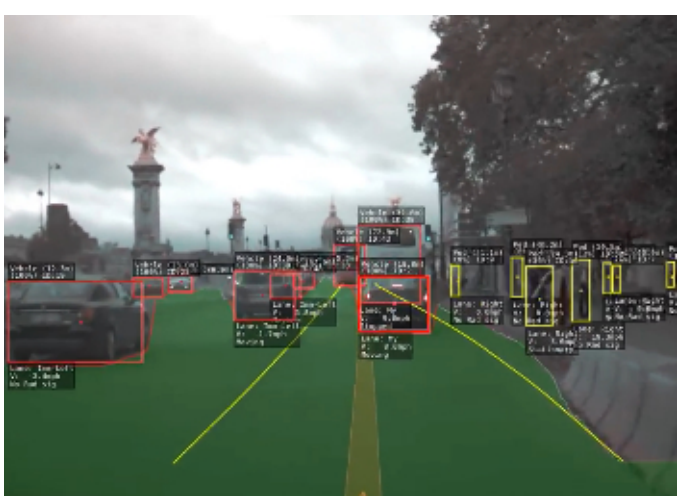
[1992]



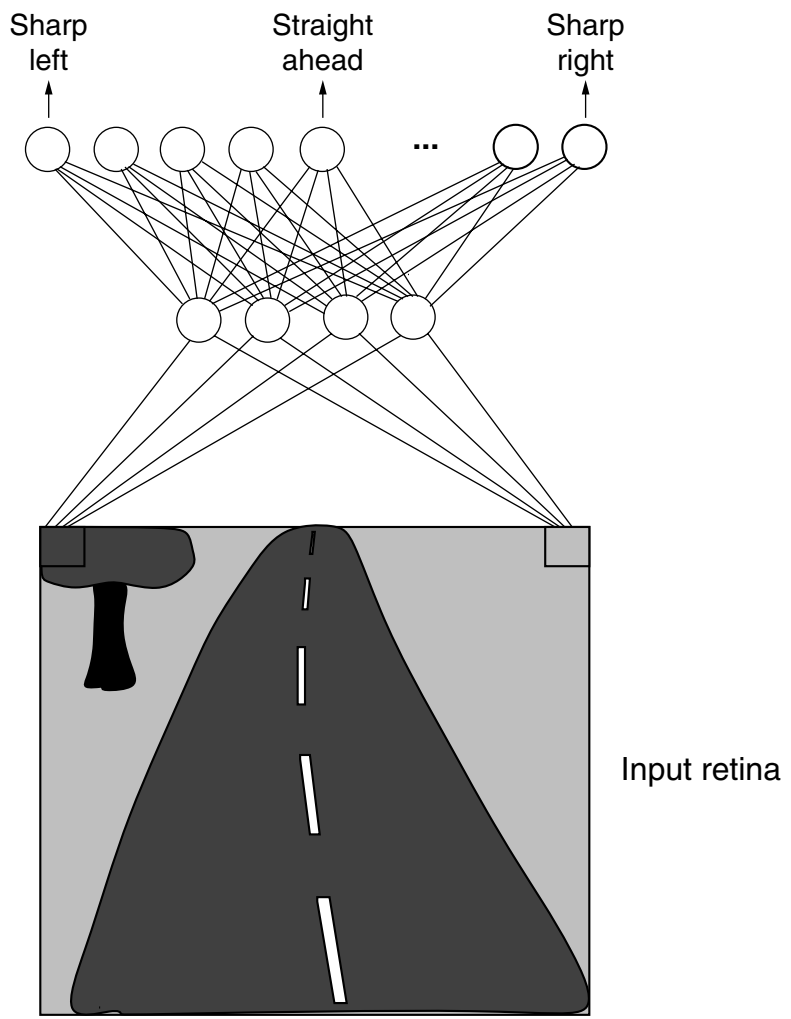
© STEIN/LETTMANN 2021

Examples of Learning Tasks

Image Analysis [Mitchell 1997]



[2018]



© STEIN/LETTMANN 2021

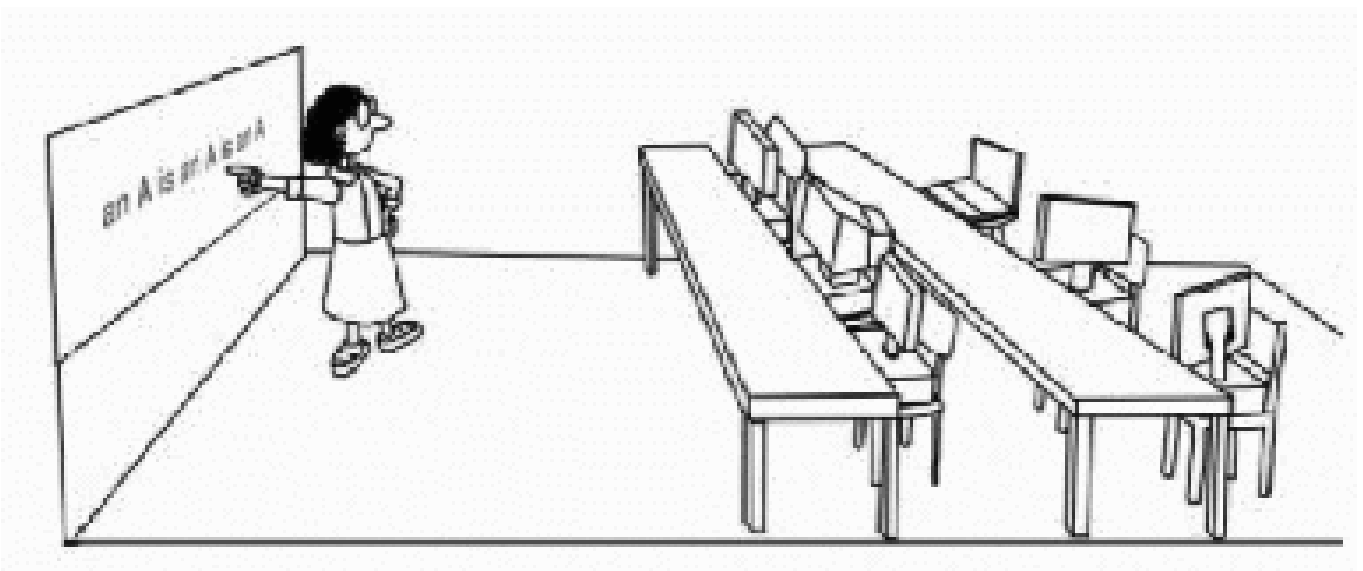
Specification of Learning Tasks

Definition 1 (Machine Learning [Mitchell 1997]**)**

A computer program is said to learn

- from experience
- with respect to some class of tasks and
- a performance measure,

if its performance at the tasks improves with the experience.



© STEIN/LETTMANN 2021

Remarks:

- Example: chess
  - task = playing chess
  - performance measure = number of games won during a world championship
  - experience = possibility to play against itself
- Example: optical character recognition
  - task = isolation and classification of handwritten words in bitmaps
  - performance measure = percentage of correctly classified words
  - experience = collection of correctly classified, handwritten words
- A data set (a corpus) with labeled examples forms a kind of "compiled experience".
- Consider the different data sets that are developed and exploited for different learning tasks in the Webis Group. [webis.de/data.html]

© STEIN/LETTMANN 2021

Specification of Learning Tasks

Learning Paradigms

1. Supervised learning

   Learn a function from a set of input-output-pairs. An important branch of supervised learning is automated classification.

   Example: optical character recognition

2. Unsupervised learning

   Identify structures in data. Important subareas of unsupervised learning include automated categorization (e.g. via cluster analysis), parameter optimization (e.g. via expectation maximization), and feature extraction (e.g. via factor analysis).

   Example: intrusion detection in a network data stream

3. Reinforcement learning

   Learn, adapt, or optimize a behavior strategy in order to maximize the own benefit by interpreting feedback that is provided by the environment.

   Example: development of behavior strategies in a hostile environment.

© STEIN/LETTMANN 2021

Specification of Learning Tasks

Learning Paradigms

1. Supervised learning

   Learn a function from a set of input-output-pairs. An important branch of supervised learning is automated classification.

   Example: optical character recognition

2. Unsupervised learning

   Identify structures in data. Important subareas of unsupervised learning include automated categorization (e.g. via cluster analysis), parameter optimization (e.g. via expectation maximization), and feature extraction (e.g. via factor analysis).

   Example: intrusion detection in a network data stream

3. Reinforcement learning

   Learn, adapt, or optimize a behavior strategy in order to maximize the own benefit by interpreting feedback that is provided by the environment.

   Example: development of behavior strategies in a hostile environment.

© STEIN/LETTMANN 2021

Specification of Learning Tasks

Example Chess: Kinds of Experience [Mitchell 1997]

1. Feedback
   - direct: for each board configuration the best move is given.
   - indirect: only the final result is given after a series of moves.

© STEIN/LETTMANN 2021

Specification of Learning Tasks

Example Chess: Kinds of Experience [Mitchell 1997]

1. Feedback
   - direct: for each board configuration the best move is given.
   - indirect: only the final result is given after a series of moves.

2. Sequence and distribution of examples
   - A teacher presents important example problems along with a solution.
   - The learner chooses from the examples; e.g., picks a board for which the best move is unknown.

The selection of examples to learn from should follow the (expected) distribution of future problems.

© STEIN/LETTMANN 2021

Specification of Learning Tasks

Example Chess: Kinds of Experience [Mitchell 1997]

1. Feedback
   - direct: for each board configuration the best move is given.
   - indirect: only the final result is given after a series of moves.

2. Sequence and distribution of examples
   - A teacher presents important example problems along with a solution.
   - The learner chooses from the examples; e.g., picks a board for which the best move is unknown.

   The selection of examples to learn from should follow the (expected) distribution of future problems.

3. Relevance under a performance measure
   - How far can we get with experience?
   - Can we master situations in the wild?
     (playing against itself may not be enough to become world class)

©STEIN/LETTMANN 2021

Specification of Learning Tasks

Example Chess: Ideal Target Function $\gamma(o)$ [Mitchell 1997]

a) γ : *Boards* $\rightarrow$ *Moves*

b) γ : *Boards* $\rightarrow \mathbf{R}$

© STEIN/LETTMANN 2021

Specification of Learning Tasks

Example Chess: Ideal Target Function $\gamma(o)$ [Mitchell 1997]

a) $\gamma : \textit{Boards} \rightarrow \textit{Moves}$

b) $\gamma : \textit{Boards} \rightarrow \mathbf{R}$

A recursive definition of $\gamma(o)$, following a kind of *means-end analysis*:

Let be $o \in \textit{Boards}$.

1. $\gamma(o) = 100$, if o represents a final board state that is won.
2. $\gamma(o) = -100$, if o represents a final board state that is lost.
3. $\gamma(o) = 0$, if o represents a final board state that is drawn.
4. $\gamma(o) = \gamma(o^*)$ otherwise.

o^* denotes the best final state that can be reached if both sides play optimally.
Related: minimax strategy, α-β pruning. [Course on Search Algorithms]

©STEIN/LETTMANN 2021

Specification of Learning Tasks

Example Chess: Real World $\rightarrow$ Model World

$$\gamma(o) \;\rightarrow\; y\big(\alpha(o)\big) \;\equiv\; y(\mathbf{x})$$

"model function"

"model formation function"

©STEIN/LETTMANN 2021

Specification of Learning Tasks

Example Chess: Real World → Model World

$\gamma(o) \;\rightarrow\; y\big(\alpha(o)\big) \;\equiv\; y(\mathbf{x})$ "model function"

"model formation function"

$$y(\mathbf{x}) = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + w_4 \cdot x_4 + w_5 \cdot x_5 + w_6 \cdot x_6$$

where

$x_{1,\,2}$ = number of black / white pawns on board o

$x_{3,\,4}$ = number of black / white pieces on board o

$x_{5,\,6}$ = number of black / white pieces threatened on board o

D = $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$, a set of board descriptions $\mathbf{x}_i$ with scores $y_i, y_i \in [-100; 100]$.

© STEIN/LETTMANN 2021

Specification of Learning Tasks

Example Chess: Real World → Model World

$\gamma(o) \;\rightarrow\; y\big(\alpha(o)\big) \;\equiv\; y(\mathbf{x})$ "model function"

"model formation function"

$$y(\mathbf{x}) = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + w_4 \cdot x_4 + w_5 \cdot x_5 + w_6 \cdot x_6$$

where

$x_{1,\,2}$ = number of black / white pawns on board o

$x_{3,\,4}$ = number of black / white pieces on board o

$x_{5,\,6}$ = number of black / white pieces threatened on board o

D = $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$, a set of board descriptions $\mathbf{x}_i$ with scores $y_i, y_i \in [-100; 100]$.

Other approaches to specify a model function:

- case base
- set of rules
- neural network
- higher order polynomial of board features

©STEIN/LETTMANN 2021

Remarks:

- The *ideal target function* $\gamma(o)$ interprets the real world, say, a real-world object o, to "compute" $\gamma(o)$. This "computation" can be operationalized by a human or by some other (even arcane) mechanism of the real world.
- To simulate the interesting aspects of the real world by means of a computer, we consider a model world. This model world is restricted to particular—typically easily measurable—features $\mathbf{x}$ that are derived from o, with $\mathbf{x} = \alpha(o)$.
  In the model world, $y(\mathbf{x})$ is the abstracted and formalized counterpart of $\gamma(o)$.
- $y(\mathbf{x})$ is called *model function* or *model*. The choice and computation of a suited model function is a central aspect in the field of machine learning.
- $\alpha(o)$ is called *model formation function*. The development of a suited model formation function is often not treated as part of machine learning but "outsourced" to respective domain experts. However, tackling advanced learning tasks such as autonomous driving, automated debating, or playing chess requires a tight cooperation between the developers of $\alpha(o)$ and $y(\mathbf{x})$.
- The key difference between an ideal target function $\gamma(o)$ and a model function $y(\mathbf{x})$ lies in the complexity and the representation of their respective domains. Examples:
  - A chess grand master assesses a board o in its entirety, both intuitively and analytically; a chess program is restricted to particular features $\mathbf{x}$, $\mathbf{x} = \alpha(o)$.
  - A human mushroom picker assesses a mushroom o with all her skills (intuitively, analytically, by tickled senses); a classification program is restricted to a few surface features $\mathbf{x}$, $\mathbf{x} = \alpha(o)$.

© STEIN/LETTMANN 2021

Remarks (continued):

- For automated chess playing a real-valued assessment function is needed; such kind of tasks form regression problems. If only a small number of values are to be considered (e.g. school grades), we are given a classification problem. A regression problem can be transformed into a classification problem via domain discretization.
- Regression problems and classification problems differ in the way how an achieved accuracy or goodness of fit is assessed. For regression problems the sum of the squared residuals, RSS, may be a sensible criterion; for classification problems the number of misclassified examples may be more relevant. Keywords: *regression loss* versus *classification loss*
- For classification problems, the ideal target function $\gamma(o)$ is also called ideal *classifier*; analogously, the model function $y(\mathbf{x})$ is called classifier.
- Decision problems are classification problems with two classes.
- The halting problem for Turing machines is an undecidable classification problem.

© STEIN/LETTMANN 2021

Specification of Learning Tasks [model world]

Real World $\rightarrow$ Model World

Setting of the real world:

- O is a set of objects. (example: emails)
- C is a set of classes. (example: spam versus ham)
- $\gamma : O \rightarrow C$ is the ideal classifier for O. (γ is a human expert)

Classification task:

- Given some $o \in O$, determine its class $\gamma(o) \in C$. (example: is an email spam?)

© STEIN/LETTMANN 2021

Specification of Learning Tasks [model world]

Real World → Model World

Setting of the real world:

- O is a set of objects. (example: emails)
- C is a set of classes. (example: spam versus ham)
- $\gamma : O \to C$ is the ideal classifier for O. (γ is a human expert)

Classification task:

- Given some $o \in O$, determine its class $\gamma(o) \in C$. (example: is an email spam?)

Acquisition of classification knowledge D:

1. Collect real-world examples of the form $(o, \gamma(o))$, $o \in O$.
2. Abstract the objects towards feature vectors $\mathbf{x} \in X$, where $\mathbf{x} = \alpha(o)$.
3. Construct examples as $(\mathbf{x}, c(\mathbf{x}))$, where $\mathbf{x} = \alpha(o)$ and $c(\mathbf{x}) \equiv \gamma(o)$.

© STEIN/LETTMANN 2021

Specification of Learning Tasks [real world]

Real World $\rightarrow$ Model World (continued)

Setting of the model world:

- X is a set of feature vectors. (example: word frequency vectors)
- C is a set of classes. (as before: spam versus ham)
- $c : X \rightarrow C$ is the ideal classifier for X. (c is unknown)
- $D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times C$ is a set of examples.

Todo:

- Approximate $c(\mathbf{x})$, which is implicitly given via D, by a model function $y(\mathbf{x})$.

© STEIN/LETTMANN 2021

Specification of Learning Tasks [real world]

Real World $\rightarrow$ Model World (continued)

Setting of the model world:

- X is a set of feature vectors. (example: word frequency vectors)
- C is a set of classes. (as before: spam versus ham)
- $c : X \rightarrow C$ is the ideal classifier for X. (c is unknown)
- $D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times C$ is a set of examples.

Todo:

- Approximate $c(\mathbf{x})$, which is implicitly given via D, by a model function $y(\mathbf{x})$.

Machine learning:

1. Formulate a model function $y : X \rightarrow C, \ \mathbf{x} \mapsto y(\mathbf{x})$ (y needs to be fitted)
2. Apply statistics, theory, and algorithms from the field of machine learning to maximize the goodness of fit between the functions $c(\mathbf{x})$ and $y(\mathbf{x})$.

© STEIN/LETTMANN 2021

Specification of Learning Tasks

Real World $\rightarrow$ Model World (continued)

$$\underset{O}{\text{Objects}} \xrightarrow{\gamma(o)} \underset{C}{\text{Classes}}$$

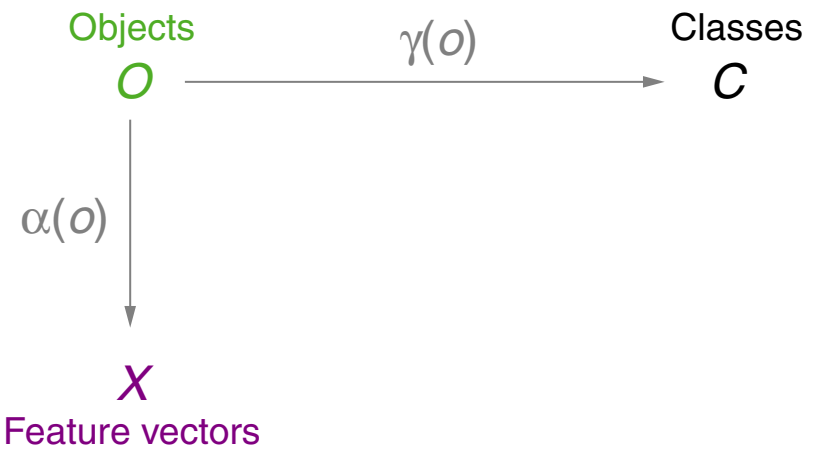
Semantics:

$\gamma(o)$ Ideal classifier (a human) for real-world objects.

©STEIN/LETTMANN 2021

Specification of Learning Tasks

Real World → Model World (continued)



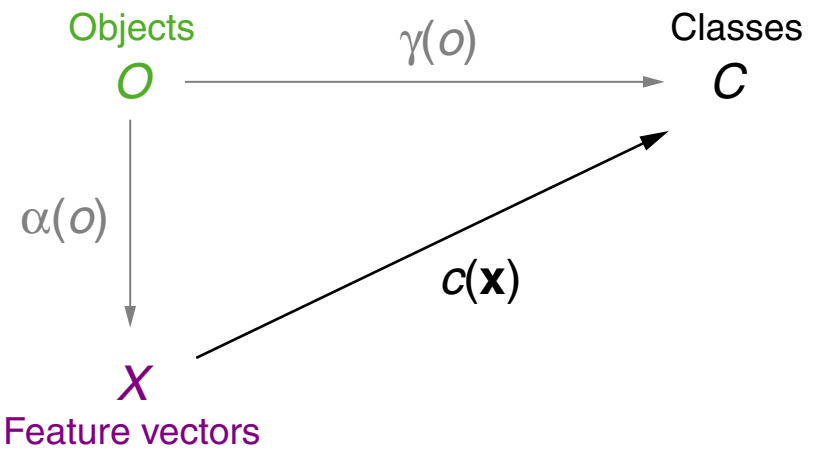
Semantics:

$\gamma(o)$	Ideal classifier (a human) for real-world objects.
$\alpha(o)$	Model formation function.

© STEIN/LETTMANN 2021

Specification of Learning Tasks

Real World → Model World (continued)



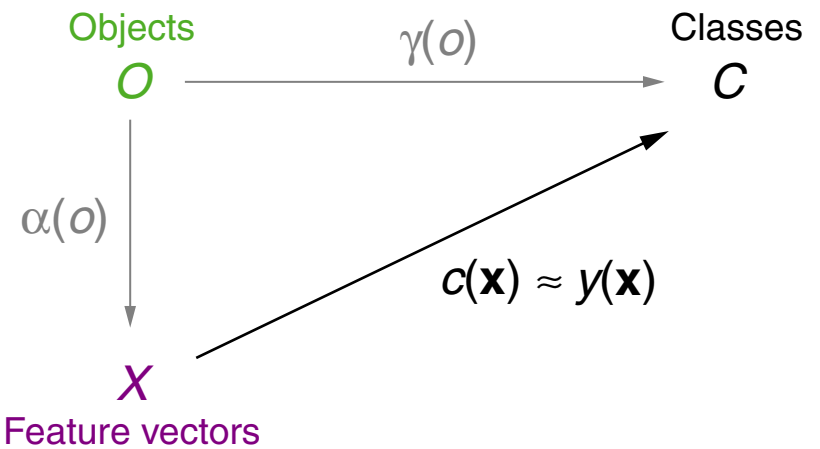
Semantics:

$\gamma(o)$	Ideal classifier (a human) for real-world objects.
$\alpha(o)$	Model formation function.
$c(\mathbf{x})$	Unknown ideal classifier for feature vectors.

© STEIN/LETTMANN 2021

Specification of Learning Tasks

Real World → Model World (continued)



Semantics:

$\gamma(o)$	Ideal classifier (a human) for real-world objects.
$\alpha(o)$	Model formation function.
$c(\mathbf{x})$	Unknown ideal classifier for feature vectors.
$y(\mathbf{x})$	Classifier (model function) to be learned.
$c(\mathbf{x}) \approx y(\mathbf{x})$	$c(\mathbf{x})$ is approximated by $y(\mathbf{x})$, based on a set of examples D.

©STEIN/LETTMANN 2021

Remarks:

- Feature vectors $\mathbf{x}_1, \mathbf{x}_2, \ldots$, are abstractions of real-world objects $o_1, o_2, \ldots$ While the objects $\mathbf{o}$ may be considered as unique, the feature vectors $\mathbf{x}$ are typically not since similar objects may be mapped onto the same feature vector. I.e, the set X actually is a multiset, comprised of feature vectors resulting from the analysis of a sequence of unique real-world objects.
- To be distinguished from the set X of feature vectors is the space $\mathbf{X}$, which is the Cartesian product of the domains of the different features (dimensions) of a feature vector $\mathbf{x}$. In the chess example $\mathbf{X}$ is the $\mathbf{N}^6$, and, in the upcoming regression settings $\mathbf{X}$ is the $\mathbf{R}^p$.
- $\mathbf{X}$ is called input space, feature space, feature domain, domain, or similar. The structure of the feature space (consider a pre-Hilbert space, a σ-algebra, or even no structure) is an important determinant of the learning problem.
- Heads-up: We will distinguish carefully between the two terms input space and feature space if an explicit feature transformation step is done such as basis expansion (regression), multilayer perceptron processing (neural networks), or non-linear kernelization (support vector machines).

© STEIN/LETTMANN 2021

Remarks (continued):

- The model formation function $\alpha(o)$ determines the level of abstraction between o and $\mathbf{x}$, $\mathbf{x} = \alpha(o)$. I.e., $\alpha(o)$ determines the representation fidelity, exactness, quality, or simplification.
- Though $\alpha(o)$ models an object $o \in O$ only imperfectly as $\mathbf{x} = \alpha(o)$, $c(\mathbf{x})$ must be considered as *ideal* classifier, since $c(\mathbf{x})$ is defined as $\gamma(o)$, the true real-world class. I.e., $c(\mathbf{x})$ and $\gamma(o)$ have different domains each, but they return the same images.
- The function $c(\mathbf{x})$ is only implicitly given, in the form of the example set D. The representation of c as a closed function is unknown, it is approximated by y.
- $c(\mathbf{x})$ is often termed "ground truth" (for $\mathbf{x}$ and the underlying classification problem). Observe that this term is justified by the fact that $c(\mathbf{x}) \equiv \gamma(o)$.
- Note that in the chess example $\gamma(o)$ defines the scores y_i only for three types of boards (won, lost, drawn). $\gamma(o)$ is unknown for all boards o that fall under Point (4) of the recursive definition. I.e., for most chess boards o we *cannot provide the ground truth* $\gamma(o)$, say, we can neither give a statement whether o leads to a final board state that is won or lost if both sides play optimally nor provide the next optimum move.

© STEIN/LETTMANN 2021

Specification of Learning Tasks

The LMS Algorithm for Fitting $y(\mathbf{x})$

[Algorithms: LMS BGD$_\sigma$ PT BGD$_2$ IGD]

Algorithm:	LMS	Least Mean Squares.
Input:	D	Training examples $(\mathbf{x}, c(\mathbf{x}))$ with $\mathbf{x} \in \mathbf{R}^p$ and target class $c(\mathbf{x})$.
	η	Learning rate, a small positive constant.
Output:	$\mathbf{w}$	Weight vector from $\mathbf{R}^{p+1}$ (= hypothesis).

LMS(D, η)

1. *initialize_random_weights*$((w_0, w_1, \ldots, w_p))$
2. **REPEAT**
3. $(\mathbf{x}, c(\mathbf{x})) =$ *random_select*(D)
4. $y(\mathbf{x}) = w_0 + w_1 \cdot x_1 + \ldots + w_p \cdot x_p \overset{(\star)}{=} \mathbf{w}^T\mathbf{x}$ // $(\star)$ With $\mathbf{x} = (1, x_1, \ldots, x_p)^T \in \mathbf{R}^{p+1}$.
5. $\delta = c(\mathbf{x}) - y(\mathbf{x})$
6. $\Delta\mathbf{w} \overset{(\star)}{=} \eta \cdot \delta \cdot \mathbf{x}$ // $\delta \cdot \mathbf{x}$ is the derivative of $\ell_2(c(\mathbf{x}), y(\mathbf{x}))$.
7. $\mathbf{w} = \mathbf{w} + \Delta\mathbf{w}$
8. **UNTIL**(*convergence*$(c(D), y(D))$)
9. *return*$((w_0, w_1, \ldots, w_p))$

©STEIN/LETTMANN 2021

Remarks:

- Line 6: The derivative of the pointwise squared loss $\ell_2(c(\mathbf{x}), y(\mathbf{x}))$ is used to update the weight vector. See section Gradient Descent of part Neural Networks for a derivation of the update term.

- Line 8: The *convergence* function may analyze the global loss quantified as sum of squared residuals, $\sum_{(\mathbf{x},c(\mathbf{x}))\in D} (c(\mathbf{x}) - y(\mathbf{x}))^2$, or check an upper bound on the number of iterations. In this regard $c(D)$ and $y(D)$ denote the vectors of target classes and computed values respectively.

© STEIN/LETTMANN 2021

Remarks (continued):

- The LMS algorithm approximates the solution of a least squares problem. It is a specialization of the SGD algorithm (stochastic gradient descent)—more specifically, an SGD that uses a squared loss (cost) function.

  The SGD algorithm, in turn, is a stochastic approximation of gradient descent optimization; it replaces the true gradient (calculated from the entire data set) by an estimate thereof (calculated from a randomly selected subset of the data).

- The LMS algorithm is hence very similar to the IGD algorithm (incremental gradient descent) equipped with a squared loss (cost) function. They differ in their selection of examples:

  [LMS] Line 3: $(\mathbf{x}, c(\mathbf{x})) = \textit{random_select}(D)$

  [IGD] Line 4: FOREACH $(\mathbf{x}, c(\mathbf{x})) \in D$ DO

  The IGD algorithm approximates the global direction of steepest loss descent as used by the BGD algorithm (batch gradient descent), for which more rigorous statements on convergence are possible.

© STEIN/LETTMANN 2021